

I. Objetivos

Al completar esta guía, el estudiante será capaz de:

- Comprender la diferencia entre estructuras de datos estáticas y dinámicas.
- Declarar e inicializar arreglos unidimensionales y bidimensionales en C#.
- Manipular datos dentro de arreglos.
- Crear y utilizar arreglos de objetos en C#.
- Identificar escenarios de aplicación de arreglos en el desarrollo de software.

II. Introducción Teórica

Para entender los arreglos, primero debemos hablar de **estructuras de datos**.

a. Estructuras de Datos

En programación, una **estructura de datos** es una forma particular de organizar y almacenar datos en una computadora para que puedan ser utilizados de manera eficiente. Piensen en ellas como contenedores especializados para guardar información.

b. Estructuras de Datos Estáticas

Las **estructuras de datos estáticas** son aquellas cuyo tamaño es fijo y se define en tiempo de compilación. Una vez declaradas, su tamaño no puede cambiar durante la ejecución del programa.

i. Principales Usos

- Cuando el número de elementos a almacenar es conocido y no va a cambiar.
- Optimización de memoria en escenarios donde se necesita un tamaño constante.
- Implementación de algoritmos que requieren acceso directo a elementos por índice.

c. Estructuras de Datos Dinámicas

A diferencia de las estáticas, las **estructuras de datos dinámicas** pueden cambiar su tamaño durante la ejecución del programa. Pueden crecer o encogerse según las necesidades de almacenamiento.

i. Principales Usos

- Cuando el número de elementos a almacenar es desconocido o varía significativamente.
- Cuando se necesita flexibilidad para añadir o eliminar elementos con frecuencia.
- Ejemplos comunes incluyen listas enlazadas, árboles y colas.

Ahora, enfoquémonos en los **arreglos**, que son un tipo fundamental de estructura de datos estática.

d. Sintaxis de Arreglos Unidimensionales en C#

Un **arreglo unidimensional** (también conocido como vector) es una colección de elementos del mismo tipo, almacenados en ubicaciones de memoria contiguas y accesibles mediante un índice.

Sintaxis de declaración e inicialización:

Declaración sin inicialización

tipoDato[] nombreArreglo;

Declaración e inicialización con tamaño

tipoDato[] nombreArreglo = new tipoDato[tamaño];

Declaración e inicialización con valores directamente

tipoDato[] nombreArreglo = {valor1, valor2, ..., valorN};

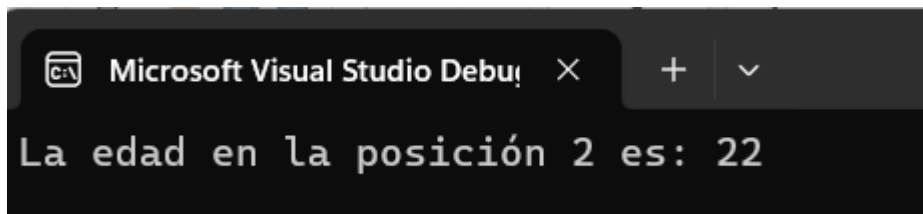
tipoDato[] nombreArreglo = new tipoDato[] {valor1, valor2, ..., valorN};

i. Ejemplos

```
// Ejemplo 1: Arreglo de enteros
int[] edades = new int[5]; // Declara un arreglo de 5 enteros
edades[0] = 25;
edades[1] = 30;
edades[2] = 22;
edades[3] = 45;
edades[4] = 18;

Console.WriteLine("La edad en la posición 2 es: " + edades[2]);
```

Generando esta salida



```
// Ejemplo 2: Arreglo de cadenas inicializado directamente
string[] nombres = { "Ana", "Luis", "Carlos", "Marta" };

Console.WriteLine("El segundo nombre es: " + nombres[1]); // Salida: Luis

// Recorriendo un arreglo con un bucle for
for (int i = 0; i < nombres.Length; i++)
{
    Console.WriteLine($"Nombre: {nombres[i]}");
}

// Recorriendo un arreglo con un bucle foreach
foreach (string nombre in nombres)
{
    Console.WriteLine($"Nombre (foreach): {nombre}");
}
```

```
Microsoft Visual Studio Debug Console
El segundo nombre es: Luis
Nombre: Ana
Nombre: Luis
Nombre: Carlos
Nombre: Marta
Nombre (foreach): Ana
Nombre (foreach): Luis
Nombre (foreach): Carlos
Nombre (foreach): Marta
```

e. Sintaxis de Arreglos Bidimensionales en C#

Un **arreglo bidimensional** (o matriz) es una colección de elementos del mismo tipo, organizados en filas y columnas.

Sintaxis de declaración e inicialización:

Declaración sin inicialización

***tipoDato[,]* nombreMatriz;**

Declaración e inicialización con filas y columnas

***tipoDato[,]* nombreMatriz = new tipoDato[numFilas, numColumnas];**

Declaración e inicialización con valores directamente

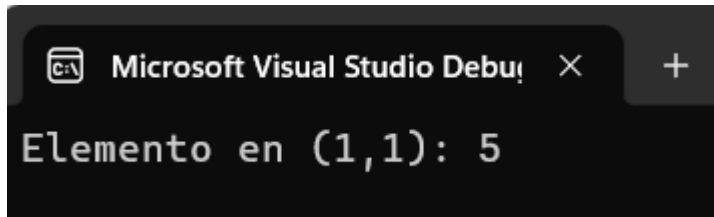
***tipoDato[,]* nombreMatriz = { {valorF1C1, valorF1C2}, {valorF2C1, valorF2C2} };**

i. Ejemplos

```
// Ejemplo 1: Matriz de enteros 3x3
int[, ] matrizNumeros = new int[3, 3];
```

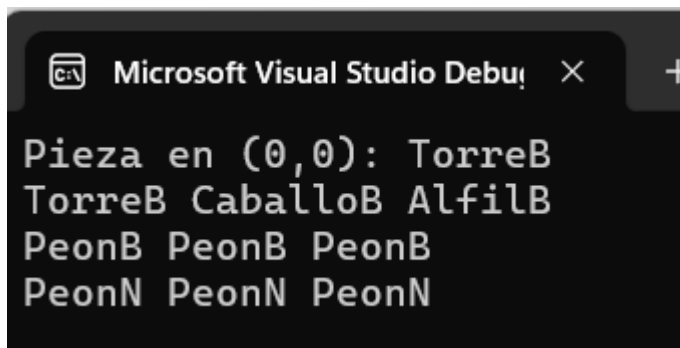
```
// Asignando valores
matrizNumeros[0, 0] = 1;
matrizNumeros[0, 1] = 2;
matrizNumeros[0, 2] = 3;
matrizNumeros[1, 0] = 4;
matrizNumeros[1, 1] = 5;
matrizNumeros[1, 2] = 6;
matrizNumeros[2, 0] = 7;
matrizNumeros[2, 1] = 8;
matrizNumeros[2, 2] = 9;
```

```
Console.WriteLine("Elemento en (1,1): " + matrizNumeros[1, 1]);
```



```
// Ejemplo 2: Matriz de cadenas inicializada
string[,] tableroAjedrez = {{"TorreB", "CaballoB", "AlfilB"},
                             {"PeonB", "PeonB", "PeonB"},
                             {"PeonN", "PeonN", "PeonN"}};

Console.WriteLine("Pieza en (0,0): " + tableroAjedrez[0, 0]); // Salida: TorreB
// Recorriendo una matriz con bucles anidados
for (int i = 0; i < tableroAjedrez.GetLength(0); i++) // GetLength(0) para filas
{
    for (int j = 0; j < tableroAjedrez.GetLength(1); j++) // GetLength(1) para columnas
    {
        Console.Write($"{tableroAjedrez[i, j]} ");
    }
    Console.WriteLine();
}
```



f. Arreglos de Objetos

Los arreglos no solo pueden almacenar tipos de datos primitivos (int, string, bool, etc.), sino también **objetos** de clases que ustedes mismos definan. Esto es extremadamente útil para manejar colecciones de entidades más complejas.

Para hacer esto, primero deben definir una clase.

i. Ejemplos

Primero, definamos una clase Estudiante:

```
public class Estudiante
{
    public string Nombre { get; set; }
    public int Edad { get; set; }
    public string Carrera { get; set; }

    public Estudiante(string nombre, int edad, string carrera)
    {
        Nombre = nombre;
    }
}
```

```

        Edad = edad;
        Carrera = carrera;
    }
    public void MostrarInformacion()
    {
        Console.WriteLine($"Nombre: {Nombre}, Edad: {Edad}, Carrera: {Carrera}");
    }
}

```

Ahora, podemos crear un arreglo de objetos Estudiante:

```

// Crear un arreglo para 3 objetos Estudiante
Estudiante[] grupoEstudiantes = new Estudiante[3];

// Instanciar objetos y asignarlos al arreglo
grupoEstudiantes[0] = new Estudiante("Juan Perez", 20, "Ingeniería de Software");
grupoEstudiantes[1] = new Estudiante("Maria Lopez", 21, "Diseño Gráfico");
grupoEstudiantes[2] = new Estudiante("Pedro Ramirez", 19, "Contaduría Pública");

// Acceder y usar los objetos en el arreglo
Console.WriteLine("Información de los estudiantes:");
foreach (Estudiante estudiante in grupoEstudiantes)
{
    estudiante.MostrarInformacion();
}

// Acceder a un atributo específico de un objeto en el arreglo
Console.WriteLine($"La carrera del primer estudiante es: {grupoEstudiantes[0].Carrera}");

```

```

Microsoft Visual Studio Debug Console
+ -

Información de los estudiantes:
Nombre: Juan Perez, Edad: 20, Carrera: Ingeniería de Software
Nombre: Maria Lopez, Edad: 21, Carrera: Diseño Gráfico
Nombre: Pedro Ramirez, Edad: 19, Carrera: Contaduría Pública
La carrera del primer estudiante es: Ingeniería de Software

```

III. Requerimientos

Nº	Cantidad	Descripción
1	1	Visual Studio 2022 Versión Community o superior
2	1	Guía Arreglos
3	1	Memoria USB o almacenamiento en la nube.

IV. Procedimiento

Ejemplo No 1

Se pide crear una solución que lea una “x” cantidad de valores numéricos del tipo entero, se almacenen en un vector de “x”. Se pide determinar en qué posición se encuentra el número primo con mayor cantidad de dígitos pares.

Controles necesarios:

Control	Propiedades sugeridas
Label	Text = "Ingrese la cantidad de números (X)"
NumericUpDown	Name = txtCantidad
Button	Name = btnGenerar, Text = "Generar"
DataGridView	Name = dgvNumeros
Button	Name = btnProcesar, Text = "Procesar"
Label	Name = lblResultado, AutoSize = true text = ""

Variable global

```
int[] numeros;
```

Evento del Form1_Load

```
private void Form1_Load(object sender, EventArgs e){
    lblResultado.Text = "";
    // Oculta la última fila vacía
    dgvNumeros.AllowUserToAddRows = false;

    // Oculta encabezado lateral de filas
    dgvNumeros.RowHeadersVisible = false;

    // Ajusta las columnas para que llenen el ancho del grid
    dgvNumeros.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;

    // (Opcional) Selección de fila completa y solo lectura
    dgvNumeros.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
    dgvNumeros.ReadOnly = true;
}
```

Evento del botón generar

```
private void btnGenerar_Click(object sender, EventArgs e)
{
    if (int.TryParse(txtCantidad.Text, out int cantidad) && cantidad > 0)
    {
        numeros = new int[cantidad];
        Random rnd = new Random();

        dgvNumeros.Rows.Clear();
        dgvNumeros.Columns.Clear();

        dgvNumeros.Columns.Add("Pos", "Posición");
        dgvNumeros.Columns.Add("Valor", "Valor");
        dgvNumeros.Columns.Add("EsPrimo", "¿Primo?");
        dgvNumeros.Columns.Add("DigitosPares", "Dígitos Pares");

        for (int i = 0; i < cantidad; i++)
        {
            numeros[i] = rnd.Next(10, 1000);
            bool esPrimo = EsPrimo(numeros[i]);
            int pares = ContarDigitosPares(numeros[i]);

            dgvNumeros.Rows.Add(i, numeros[i], esPrimo ? "Sí" : "No", pares);
        }
        // Deselecciona cualquier fila seleccionada por defecto
        dgvNumeros.ClearSelection();
        lblResultado.Text = "Datos generados. Presione 'Procesar'.";
    }
}
```

```

        else
        {
            MessageBox.Show("Ingrese un valor válido para la cantidad.");
        }
    }
}

```

Funciones auxiliares

```

private void btnProcesar_Click_1(object sender, EventArgs e)
{
    if (numeros == null || numeros.Length == 0)
    {
        MessageBox.Show("Primero debe generar los datos.");
        return;
    }

    int maxPares = -1;
    int posicionMax = -1;

    for (int i = 0; i < numeros.Length; i++)
    {
        if (EsPrimo(numeros[i]))
        {
            int pares = ContarDigitosPares(numeros[i]);
            if (pares > maxPares)
            {
                maxPares = pares;
                posicionMax = i;
            }
        }
    }

    if (posicionMax >= 0)
    {
        lblResultado.Text = $"Posición del primo con más dígitos pares: {posicionMax}";
        (Valor: {numeros[posicionMax]});
        dgvNumeros.Rows[posicionMax].DefaultCellStyle.BackColor = Color.LightGreen;
    }
    else
    {
        lblResultado.Text = "No se encontraron números primos en el vector.";
    }
}

```

Funciones auxiliares

```

// Función auxiliar: Verifica si un número es primo
private bool EsPrimo(int n)
{
    if (n <= 1) return false;
    if (n == 2) return true;
    if (n % 2 == 0) return false;

    for (int i = 3; i <= Math.Sqrt(n); i += 2)
    {
        if (n % i == 0) return false;
    }
}

```



```

    }

    return true;
}

// Función auxiliar: Cuenta dígitos pares de un número
private int ContarDigitosPares(int numero)
{
    int contador = 0;

    while (numero > 0)
    {
        int digito = numero % 10;
        if (digito % 2 == 0) contador++;
        numero /= 10;
    }

    return contador;
}

```

Ejemplo No 2

La cafetería de la Universidad cobra \$2.00 por una taza de café y ofrece los agregados que se muestran en el cuadro siguiente:

Producto	Precio (\$)
Crema batida	0.89
Canela	0.25
Jarabe de chocolate	0.59
Amaretto	1.50
Whisky irlandés	1.75

Diseñe la aplicación usando Windows form con c# que permita a un usuario introducir en forma continua los agregados ordenados hasta que se ingrese un valor centinela. Para listar los productos utilizar checkbox.

Después de cada artículo, despliegue su precio. Después de introducir todos los artículos, despliegue el precio total para la orden. Lleve un control de las ventas realizadas por día:

1. Cantidad de cafés por orden.
2. Monto total de cada orden.
3. Historial completo de ventas.

Para poder realizar nuestro objetivo, se va a necesitar los siguientes controles

Control	Propiedad Importante	Valor/Configuración
CheckBoxes	AutoSize	True
	Text	"Nombre (+\$precio)"

CheckBox (5 unidades) - Para los agregados:

- Crema batida (+\$0.89)
- Canela (+\$0.25)
- Jarabe de chocolate (+\$0.59)
- Amaretto (+\$1.50)
- Whisky irlandés (+\$1.75)

Estos controles se van a adicionar por medio de programación, para un mejor manejo

ListBox	Name	IstAgregados
	Height	100-150px
	SelectionMode	solo lectura
Buttons	FlatStyle	FlatStyle.Standard
Labels	Font	Arial 10pt (negrita para precios)

1. Labels (Etiquetas informativas):

- Precio base del café: \$2.00 (lblPrecioBase)
- Cantidad: 1 (lblCantidad)
- Total: \$2.00 (lblTotal)

El formulario sería algo así, los nombres de los controles siguen la norma de las primeras tres letras, tipo de control seguido de un nombre descriptivo de lo que hace el control.

El código del proyecto será el siguiente

```
public partial class Form1 : Form
{
    // Constantes y configuración
    private const double PRECIO_CAFE = 2.00;
    private const int MAX_PEDIDOS = 50; // Máximo de pedidos a registrar

    // Vectores de productos
    private readonly string[] agregados = { "Crema batida", "Canela", "Jarabe de chocolate", "Amaretto", "Whisky irlandés" };
    private readonly double[] precios = { 0.89, 0.25, 0.59, 1.50, 1.75 };
    private bool[] seleccionados = new bool[5];

    // Vectores para registro de ventas
    private int[] cantidadesCafes = new int[MAX_PEDIDOS];
    private double[] montosOrdenes = new double[MAX_PEDIDOS];
    private string[] detallesOrdenes = new string[MAX_PEDIDOS];
    private int totalPedidos = 0; // Contador de pedidos actuales

    // Variables de la orden actual
    private double precioTotal = PRECIO_CAFE;
    private int cantidad = 1;

    public Form1()
    {
        InitializeComponent();
        InicializarCheckBoxes();
        lblPrecioBase.Text = $"Precio base del café: ${PRECIO_CAFE}";
        ActualizarTotal();
    }

    private void InicializarCheckBoxes()
    {
        for (int i = 0; i < agregados.Length; i++)
        {
            var chk = new CheckBox
            {
                Text = $"{agregados[i]} (+${precios[i]})",
                AutoSize = true,
                Location = new Point(20, 80 + i * 30),
                Tag = i
            };
            chk.CheckedChanged += CheckBox_CheckedChanged;
            this.Controls.Add(chk);
        }
    }

    private void CheckBox_CheckedChanged(object sender, EventArgs e)
    {
        CheckBox chk = (CheckBox)sender;
        int index = (int)chk.Tag;
        seleccionados[index] = chk.Checked;
        CalcularTotal();
    }

    private void CalcularTotal()
```

```

{
    precioTotal = PRECIO_CAFE * cantidad;
    lstAgregados.Items.Clear();

    for (int i = 0; i < seleccionados.Length; i++)
    {
        if (seleccionados[i])
        {
            precioTotal += precios[i] * cantidad;
            lstAgregados.Items.Add($"{ agregados[i]} (${precios[i]})");
        }
    }

    ActualizarTotal();
}

private void ActualizarTotal()
{
    lblTotal.Text = $"Total: ${precioTotal.ToString("0.00")}";
    lblCantidad.Text = $"Cantidad: {cantidad}";
}

private void btnMenos_Click(object sender, EventArgs e)
{
    if (cantidad > 1)
    {
        cantidad--;
        CalcularTotal();
    }
}

private void btnFinalizar_Click(object sender, EventArgs e)
{
    if (totalPedidos >= MAX_PEDIDOS)
    {
        MessageBox.Show("Límite de pedidos alcanzado. Genere un reporte antes de  
continuar.", "Atención", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    // Registrar la orden en los vectores
    cantidadesCafes[totalPedidos] = cantidad;
    montosOrdenes[totalPedidos] = precioTotal;

    // Construir detalle del pedido
    string detalle = $"Orden #{totalPedidos + 1}: {cantidad} café(s) con ";
    bool tieneAgregados = false;

    for (int i = 0; i < seleccionados.Length; i++)
    {
        if (seleccionados[i])
        {
            detalle += $"{agregados[i]}, ";
            tieneAgregados = true;
        }
    }
}

```

```

        if (!tieneAgregados)
        {
            detalle += "ningún agregado";
        }
        else
        {
            detalle = detalle.TrimEnd(',', ' ');
        }

        detalle += $. Total: ${precioTotal.ToString("0.00")};
        detallesOrdenes[totalPedidos] = detalle;

        totalPedidos++;

        MessageBox.Show($"Orden registrada!\n{detalle}",
            "Venta completada",
            MessageBoxButtons.OK,
            MessageBoxIcon.Information);

        ReiniciarOrden();
    }

    private void ReiniciarOrden()
    {
        cantidad = 1;
        precioTotal = PRECIO_CAFE;
        for (int i = 0; i < seleccionados.Length; i++)
        {
            seleccionados[i] = false;
            foreach (Control c in this.Controls)
            {
                if (c is CheckBox && (int)((CheckBox)c).Tag == i)
                {
                    ((CheckBox)c).Checked = false;
                }
            }
        }
        lstAgregados.Items.Clear();
        ActualizarTotal();
    }

    private void btnReporte_Click(object sender, EventArgs e)
    {
        if (totalPedidos == 0)
        {
            MessageBox.Show("No hay órdenes registradas",
                "Reporte",
                MessageBoxButtons.OK,
                MessageBoxIcon.Information);

            return;
        }

        string reporte = "REPORTE DE VENTAS\n\n";
        double totalVentas = 0;
        int totalCafes = 0;

        for (int i = 0; i < totalPedidos; i++)
        {
            reporte += $"{detallesOrdenes[i]}\n";
            totalCafes += cantidadesCafes[i];
        }
    }

```

```

        totalVentas += montosOrdenes[i];
    }

    reporte += $"\nTOTAL CAFÉS VENDIDOS: {totalCafes}";
    reporte += $"\nTOTAL RECAUDADO: ${totalVentas.ToString("0.00")}";
    reporte += $"\nCapacidad restante: {MAX_PEDIDOS -
totalPedidos}/{MAX_PEDIDOS}";

    MessageBox.Show(reporte,
        "Reporte Completo",
        MessageBoxButtons.OK,
        MessageBoxIcon.Information);

    // Opcional: Resetear después del reporte
    // totalPedidos = 0;
}

private void btnMas_Click(object sender, EventArgs e)
{
    cantidad++;
    CalcularTotal();
}
}

```

V. Ejercicios complementarios

El instructor evaluará el desempeño de su trabajo dentro la clase. Estos ejercicios se evaluarán en la misma hora de clase.

Desarrollar una aplicación Windows Forms que permite gestionar notas de estudiantes usando vectores, donde el alumno deberá:

1. Implementar operaciones básicas con vectores
2. Manejar interfaz gráfica
3. Validar datos de entrada
4. Procesar información académica

Enunciado del Problema

Debes crear un sistema para que un profesor pueda:

- Registrar hasta 30 estudiantes (límite del vector)
- Guardar 3 notas por estudiante
- Calcular promedios individuales y general
- Buscar estudiantes por nombre
- Mostrar reportes estadísticos

VI. Análisis de resultados

Para la próxima semana

Una vez que hayan completado los ejercicios, reflexione sobre lo siguiente y discuta sus hallazgos. Investigar brevemente los puntos a continuación:

1. **Ventajas y Desventajas de los Arreglos:** ¿Cuáles son las principales ventajas de usar arreglos en C#? ¿Y cuáles son sus desventajas, especialmente en comparación con estructuras de datos dinámicas como `List<T>`?
2. **Length vs. GetLength():** Expliquen la diferencia entre la propiedad `Length` y el método `GetLength()` cuando se trabaja con arreglos. ¿Cuándo usarían uno u otro?
3. **Arreglos Irregulares (Jagged Arrays):** Investiguen qué son los "arreglos irregulares" o "jagged arrays" en C#. ¿Cuál es su sintaxis y para qué escenarios serían útiles? Proporcionen un pequeño ejemplo de código.
4. **Uso de Arreglos en Aplicaciones de Escritorio:** Piensen en al menos dos ejemplos concretos de cómo los arreglos podrían ser utilizados en una aplicación de escritorio real (ej., una aplicación de gestión de inventario, un juego simple, etc.). Describan brevemente el escenario.

V. Bibliografía

1. Microsoft Learn - Documentación oficial de .NET, <https://learn.microsoft.com/es-es/dotnet/> Incluye tutoriales de C#, WinForms, y controles UI.
2. Troelsen, A. (2023). Pro C# 10 with .NET 6. Apress. Libro de referencia para sintaxis avanzada, patrones de diseño y buenas prácticas.
3. Richter, J. (2023). CLR via C#. Microsoft Press. Fundamentos del runtime de .NET y gestión de memoria.
4. Stack Overflow - Comunidad .NET, <https://stackoverflow.com/questions/tagged/c%23>. Resolución de errores comunes y mejores prácticas validadas por la comunidad.
5. DevExpress - WinForms UI Best Practices, <https://www.devexpress.com/winforms/> Guías de diseño de interfaces y uso de controles avanzados.

Recursos adicionales

Extensión de Visual Studio:

[Microsoft XAML Controls Gallery](#) para inspirar diseños modernos.

Video-tutoriales:

Playlist de YouTube sobre WinForms (ej: "Canal de Programming with Mosh").

Libros:

- Troelsen, A. (2022). Pro C# 10 with .NET 6. Apress.
- Richter, J. (2020). CLR via C#. Microsoft Press.